

Machine, Data and Learning

Tutorial on Multi-Armed Bandits

April 18, 2026

Mathematical Formulation

Formally, an MAB problem is defined by a set of K independent arms (actions) denoted by

$$\mathcal{A} = \{1, 2, \dots, K\}.$$

At each discrete time step $t = 1, 2, \dots, T$:

- The agent selects an arm $a_t \in \mathcal{A}$.
- The environment dispenses a reward r_t drawn from a probability distribution P_{a_t} specific to that arm.
- The reward r_t is observed by the agent, and the goal is to maximize the cumulative reward over the horizon T .

Each arm i has an associated mean reward $\mu_i = \mathbb{E}[P_i]$. We denote the optimal arm as a^* , which has the maximum expected reward:

$$\mu^* = \max_{i \in \{1, \dots, K\}} \mu_i.$$

Greedy Algorithm

To define any bandit algorithm formally, we first define two state variables that we track for every arm $k \in \mathcal{A}$:

- $N_k(t)$: The count of how many times arm k has been selected up to time t
- $Q_k(t)$: The estimated value (sample mean) of arm k at time t

The value of an arm is defined as the arithmetic mean of all rewards received from that arm. If arm k has been pulled n times, its estimated value is:

$$Q_k = \frac{1}{n} \sum_{i=1}^n R_i$$

All algorithms we will look at today differ only in their decision rule for picking an arm, given these two variables.

Greedy Algorithm: Decision Rule

The Greedy algorithm uses the simplest possible decision rule:

At every time step t , perform a point estimate and select the action that maximises that estimate:

$$A_t \doteq \arg \max_k Q_k(t)$$

Note for students

If multiple arms have the same maximum estimated value, a tie breaking rule (usually uniform random selection) is applied.

Incremental Update Rule

In practice we never store all past rewards. We can derive an extremely efficient recursive update rule that only requires us to store Q_k and N_k .

Let Q_n be the estimate after n pulls. To calculate the new estimate after observing reward R_{n+1} :

$$Q_{n+1} = \frac{1}{n+1} (R_{n+1} + n \cdot Q_n)$$

$$Q_{n+1} = Q_n + \frac{1}{n+1} [R_{n+1} - Q_n]$$

This can be written in the general form that you will see again many times in reinforcement learning:

Universal Update Rule

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \times [\text{Target} - \text{OldEstimate}]$$

Algorithm 1: Greedy

Initialize:

- Pull each arm $k \in \mathcal{A}$ exactly once and observe reward R_k
- Set $N_k = 1$ and $Q_k = R_k$ for all arms $k \in \mathcal{A}$

Loop for $t = K + 1, K + 2, \dots, T$:

- 1 Select arm $A_t = \arg \max_k Q_k$
- 2 Observe reward R_t
- 3 Increment count: $N_{A_t} \leftarrow N_{A_t} + 1$
- 4 Update estimate: $Q_{A_t} \leftarrow Q_{A_t} + \frac{1}{N_{A_t}} (R_t - Q_{A_t})$

Why The Greedy Algorithm Fails

The fundamental mathematical problem is that the sample mean estimator $\hat{\mu}_k$ only converges to the true mean μ_k if the number of samples $n \rightarrow \infty$.

In a Greedy algorithm, n only increases for the arm that already has the highest current estimate. This creates an irreversible lockout effect:

- If the optimal arm k^* yields a few low rewards early on due to random variance, its sample mean drops
- If a suboptimal arm k' yields a few lucky high rewards, its sample mean stays higher
- The algorithm will now pick k' indefinitely. Because it never picks k^* again, n_{k^*} stays constant forever, and the error is never corrected.

Why The Greedy Algorithm Fails

Let's take the simplest possible case study. Two Bernoulli arms:

- Arm 1 (Optimal): $p_1 = 0.7$
- Arm 2 (Suboptimal): $p_2 = 0.2$

Greedy algorithm initializes by pulling each arm exactly once.

$$P(\text{Arm 1 returns 0}) = 0.3$$

$$P(\text{Arm 2 returns 1}) = 0.2$$

Failure Probability

There is a $0.3 \times 0.2 = 6\%$ chance that the Greedy algorithm will get permanently and irreversibly stuck on the bad arm forever, based only on the outcome of the first two pulls.

This probability grows rapidly as you add more arms or increase reward variance.

Why The Greedy Algorithm Fails

You can frame this as a complete failure to account for uncertainty.

A Greedy algorithm only ever looks at the point estimate of expected reward. It completely ignores the variance, and therefore the uncertainty, of that estimate.

The variance of a sample mean is:

$$\text{Var}(\hat{\mu}) = \frac{\sigma^2}{n}$$

If n is small, our confidence in $\hat{\mu}$ is extremely low.

The fatal flaw

The Pure Greedy algorithm treats an estimate based on 1 sample with exactly the same confidence as an estimate based on 1000 samples.

It has no concept of the value of information.

The ϵ -Greedy algorithm is the simplest possible fix to the Greedy lockout problem. Notice that every single part of this algorithm is identical to the pure Greedy algorithm, except for the decision rule.

Algorithm 2: ϵ -Greedy

- Set $Q_k = 0$ and $N_k = 0$ for all arms $k \in \mathcal{A}$
- Set hyperparameter $\epsilon \in [0, 1]$

Loop for $t = 1, 2, \dots, T$:

- 1 Generate random number $p \sim \text{Uniform}(0, 1)$
- 2 Select action A_t :
 - If $p > \epsilon$: $A_t = \arg \max_k Q_k$ (**Exploit**)
 - If $p \leq \epsilon$: $A_t = \text{uniform random arm}$ (**Explore**)
- 3 Observe reward R_t
- 4 Increment count: $N_{A_t} \leftarrow N_{A_t} + 1$
- 5 Update estimate: $Q_{A_t} \leftarrow Q_{A_t} + \frac{1}{N_{A_t}} (R_t - Q_{A_t})$

Regret

First we define the best possible performance achievable by any strategy. In a stochastic MAB, the optimal strategy is to always pull the optimal arm a^* :

$$\mu^* = \max_{k \in \mathcal{A}} \mu_k$$

Expected Regret is the expected total reward lost by our algorithm compared to the optimal benchmark, over T steps:

$$R_T = \mathbb{E} \left[T\mu^* - \sum_{t=1}^T r_t \right]$$

The single most important identity in all of bandit theory. Using linearity of expectation we can decompose regret completely:

$$\mathbb{E}[R_T] = \sum_{k=1}^K \Delta_k \cdot \mathbb{E}[N_k(T)]$$

Where $\Delta_k = \mu^* - \mu_k$ is the gap, the fixed permanent cost of pulling arm k once.

Pseudo regret is a cleaner version used almost exclusively in all modern proofs. Almost everyone confuses these two definitions.

While Expected Regret compares the optimal mean to the actual noisy rewards we received, Pseudo-regret $\bar{R}_T$ compares the optimal mean to the true mean of the arm we chose:

$$\bar{R}_T = T\mu^* - \sum_{t=1}^T \mu_{a_t}$$

Key Difference

Pseudo regret counts how much reward you should have lost on average, before you see the actual random reward. It completely removes the noise of individual rewards from the analysis.

Why Linear Regret Is A Complete Failure

We formally define an algorithm as no-regret if its average regret per step vanishes to zero as time goes to infinity:

$$\lim_{T \rightarrow \infty} \frac{\mathbb{E}[R_T]}{T} = 0$$

This is the mathematical definition of an algorithm that actually learns.

Regret Order	Asymptotic Behaviour
$O(T)$	$\lim_{T \rightarrow \infty} \frac{R_T}{T} = C$, a positive constant. The algorithm never stops making mistakes at a fixed rate.
$O(\log T)$	$\lim_{T \rightarrow \infty} \frac{R_T}{T} \rightarrow 0$ The algorithm converges to the optimal arm.

Why Greedy Has Linear Regret

We can now write the exact asymptotic regret of Pure Greedy explicitly.

Recall that Pure Greedy has a constant probability δ of getting permanently and irreversibly locked out of the optimal arm forever.

If it gets locked out, it will pull the suboptimal arm for every single remaining step, incurring the full gap Δ every time.

Therefore the total expected regret of Pure Greedy is:

$$\mathbb{E}[R_T] = \delta \cdot \Delta \cdot T$$

Since δ and Δ are constants, completely independent of T , the regret of Pure Greedy is exactly linear $O(T)$.

Pure Greedy is not a no-regret algorithm.

The Great Epsilon Greedy Lie

This is the most important surprise in this tutorial. Almost everyone thinks epsilon greedy fixes all the problems of pure greedy. It does not.

It fixes the permanent lockout problem. But it has exactly the same asymptotic regret order as pure greedy.

The ϵ -Greedy policy selects a random arm with a fixed probability ϵ at every single time step t , forever.

Let $\mathcal{A}_{sub}$ be the set of all suboptimal arms. The probability of picking a suboptimal arm at any time t is always at least:

$$P(A_t \in \mathcal{A}_{sub}) \geq \epsilon \cdot \frac{K-1}{K}$$

Because ϵ is a constant that does not depend on t , the agent will never stop sampling suboptimal arms, even as $t \rightarrow \infty$.

Recall the definition of pseudo regret:

$$\bar{R}_T = \sum_{t=1}^T (\mu^* - \mu_{a_t})$$

Suppose after some time the algorithm has correctly identified the optimal arm, so $\arg \max Q_k = a^*$. The expected loss at every single future timestep is:

$$\mathbb{E}[\text{loss}_t] = (1 - \epsilon) \cdot 0 + \epsilon \cdot \text{Avg}(\Delta)$$

Where $\text{Avg}(\Delta)$ is the average gap of all suboptimal arms. Since ϵ and the gaps are all constants:

$$\mathbb{E}[\text{loss}_t] = C \quad \text{where } C > 0$$

Summing this over all T steps:

$$\mathbb{E}[R_T] = \sum_{t=1}^T C = C \cdot T$$

Fixed epsilon greedy has exactly linear $O(T)$ regret. Exactly the same asymptotic order as Pure Greedy.

Dynamic ϵ -Greedy: A Natural Fix

The root cause of linear regret in fixed ϵ -Greedy is clear: we keep exploring at a constant rate ϵ even after we have confidently identified the optimal arm.

Intuition: We should explore *a lot* early on, when our estimates are uncertain, and explore *less and less* as our estimates improve.

The simplest way to implement this: let ϵ decay with time.

Algorithm 3: Dynamic ϵ_t -Greedy

Replace the fixed ϵ with a schedule ϵ_t that decreases over time. A natural choice:

$$\epsilon_t = \frac{1}{t}$$

Everything else is identical to ϵ -Greedy. At time t , explore with probability $\frac{1}{t}$ and exploit otherwise.

Regret of Dynamic $\epsilon_t = 1/t$ Greedy

Since $\epsilon_t \rightarrow 0$, it appears we explore less and less. But does this achieve sub-linear regret?

Recall the expected loss at step t (assuming the optimal arm is identified):

$$\mathbb{E}[\text{loss}_t] = \epsilon_t \cdot \text{Avg}(\Delta) = \frac{C}{t}$$

Summing over all T steps:

$$\mathbb{E}[R_T] \leq C \sum_{t=1}^T \frac{1}{t} = C \cdot H_T$$

where $H_T = \sum_{t=1}^T \frac{1}{t}$ is the T -th harmonic number.

Recall from Analysis

The harmonic series diverges logarithmically:

$$H_T = \sum_{t=1}^T \frac{1}{t} = \ln T + O(1)$$

Putting it together:

$$\mathbb{E}[R_T] = O(\log T)$$

Great news and bad news

Good: This is sub-linear. The algorithm genuinely learns. Average regret per step $\rightarrow 0$.

Bad: It is *not* the best possible $O(\log T)$. The *constant* hidden inside the $O(\cdot)$ is much larger than it needs to be.

Why is it suboptimal? The $1/t$ schedule is *oblivious* to the actual problem structure. It does not use the gaps Δ_k . It over-explores easy problems (large gaps) and under-explores hard problems (small gaps).

A principled algorithm should allocate exploration *precisely* according to how uncertain each arm estimate is, and how much that uncertainty matters.

What is the Optimal Regret?

A natural question: is $O(\log T)$ optimal, or can we do better?

Lai & Robbins (1985) Lower Bound

For *any* consistent algorithm (one that achieves sub-linear regret on every bandit instance), the expected number of times a suboptimal arm k is pulled must satisfy:

$$\liminf_{T \rightarrow \infty} \frac{\mathbb{E}[N_k(T)]}{\ln T} \geq \frac{1}{\text{KL}(\mu_k \parallel \mu^*)}$$

where $\text{KL}(\mu_k \parallel \mu^*)$ is the KL-divergence between the two arm distributions.

This gives an instance-dependent lower bound on total regret:

$$\mathbb{E}[R_T] \geq \left(\sum_{k: \Delta_k > 0} \frac{\Delta_k}{\text{KL}(\mu_k \parallel \mu^*)} \right) \ln T$$

Key Takeaway

$\Omega(\log T)$ regret is *unavoidable*. No algorithm can do better in the worst case. The goal is to match this bound, up to constant factors.

Upper Confidence Bound (UCB): Intuition

The core insight missing from both Greedy and ϵ -Greedy:

*Exploration should be directed at arms whose true value is **uncertain**, not chosen uniformly at random.*

Optimism in the face of uncertainty.

Rather than using the point estimate Q_k , UCB augments it with an *upper confidence bonus*:

$$\text{UCB}_k(t) = \underbrace{Q_k(t)}_{\text{exploitation}} + \underbrace{U_k(t)}_{\text{exploration bonus}}$$

The bonus $U_k(t)$ is designed so that with high probability:

$$\mu_k \leq Q_k(t) + U_k(t)$$

We are optimistic: we pick the arm that *could plausibly* be the best.

Key Tool: Hoeffding's Inequality

To derive the confidence bonus, we need a concentration inequality.

Hoeffding's Inequality

Let $X_1, \dots, X_n$ be i.i.d. random variables bounded in $[0, 1]$. Let $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i$. Then for any $\delta > 0$:

$$P(\hat{\mu} - \mu \geq \delta) \leq e^{-2n\delta^2}$$

and symmetrically:

$$P(\mu \geq \hat{\mu} + \delta) \leq e^{-2n\delta^2}$$

Setting the right-hand side to some small probability α and solving for δ :

$$e^{-2n\delta^2} = \alpha \implies \delta = \sqrt{\frac{\ln(1/\alpha)}{2n}}$$

This is the half-width of our confidence interval. With probability $1 - \alpha$, the true mean lies within δ of our estimate.

UCB1 Algorithm

Choose $\alpha = t^{-4}$ (we explain this choice in the proof). Then the confidence bonus for arm k at time t is:

$$U_k(t) = \sqrt{\frac{2 \ln t}{N_k(t)}}$$

Algorithm 4: UCB1 (Auer et al., 2002)

Initialize: Pull each arm exactly once (steps 1 to K).

Loop for $t = K + 1, K + 2, \dots, T$:

- 1 Select arm: $A_t = \arg \max_{k \in \mathcal{A}} \left[Q_k(t) + \sqrt{\frac{2 \ln t}{N_k(t)}} \right]$
- 2 Observe reward R_t
- 3 Update: $N_{A_t} \leftarrow N_{A_t} + 1, \quad Q_{A_t} \leftarrow Q_{A_t} + \frac{1}{N_{A_t}}(R_t - Q_{A_t})$

Notice: No ϵ hyperparameter. The algorithm is entirely self-tuning. The bonus automatically shrinks as $N_k(t)$ grows, and grows as $\ln t$ slowly increases.

Theorem (UCB1 Regret Bound)

For a K -arm bandit with rewards in $[0, 1]$, the UCB1 algorithm satisfies:

$$\mathbb{E}[R_T] \leq \sum_{k: \Delta_k > 0} \left(\frac{8 \ln T}{\Delta_k} + \left(1 + \frac{\pi^2}{3}\right) \Delta_k \right)$$

Reading this bound:

- The dominant $O(\log T)$ term is *inversely proportional* to Δ_k : arms that are hard to distinguish from the optimal arm get pulled more often.
- The constant $(1 + \pi^2/3) \approx 4.3$ is a small per-arm additive overhead.
- This matches the Lai-Robbins lower bound up to constant factors for bounded rewards.

We want to bound $\mathbb{E}[N_k(T)]$ for every suboptimal arm k .

Recall regret decomposes as:

$$\mathbb{E}[R_T] = \sum_{k: \Delta_k > 0} \Delta_k \cdot \mathbb{E}[N_k(T)]$$

Fix a suboptimal arm k with gap $\Delta_k > 0$. Let $\ell = \lceil 8 \ln T / \Delta_k^2 \rceil$ be a threshold (we will see why this choice shortly).

UCB1 selects arm k at time t only when:

$$Q_k(t) + \sqrt{\frac{2 \ln t}{N_k(t)}} \geq Q_{k^*}(t) + \sqrt{\frac{2 \ln t}{N_{k^*}(t)}}$$

We decompose this event into three “bad” events. If *none* of the three bad events holds, then arm k *cannot* be selected once $N_k(t) > \ell$.

Three Events

At time t , define the following events for the optimal arm k^* and any suboptimal arm k :

- **Event $\mathcal{E}_1$:** The UCB of the optimal arm is below its true mean.

$$\mathcal{E}_1(t) = \left\{ Q_{k^*}(t) + \sqrt{\frac{2 \ln t}{N_{k^*}(t)}} < \mu^* \right\}$$

The confidence interval fails to cover μ^* from above.

- **Event $\mathcal{E}_2$:** The estimated mean of suboptimal arm k is too high.

$$\mathcal{E}_2(t) = \left\{ Q_k(t) > \mu_k + \sqrt{\frac{2 \ln t}{N_k(t)}} \right\}$$

The confidence interval fails to cover μ_k from below.

- **Event $\mathcal{E}_3$:** Suboptimal arm k has been pulled too few times.

$$\mathcal{E}_3(t) = \{N_k(t) \leq \ell\}$$

Lemma

If none of $\mathcal{E}_1(t)$, $\mathcal{E}_2(t)$, $\mathcal{E}_3(t)$ hold, then arm k is *not* selected at time t .

Proof of Lemma. Suppose none of the three events hold. Then $N_k(t) > \ell$ and the UCB of the suboptimal arm satisfies:

$$\begin{aligned} Q_k(t) + \sqrt{\frac{2 \ln t}{N_k(t)}} &\leq \mu_k + 2\sqrt{\frac{2 \ln t}{N_k(t)}} \quad (\text{since } \mathcal{E}_2 \text{ does not hold}) \\ &< \mu_k + 2\sqrt{\frac{2 \ln t}{\ell}} \leq \mu_k + \Delta_k = \mu^* \end{aligned}$$

(using $\ell \geq 8 \ln T / \Delta_k^2$, so $2\sqrt{2 \ln t / \ell} \leq \Delta_k$).

Meanwhile, since $\mathcal{E}_1$ does not hold: $\text{UCB}_{k^*}(t) \geq \mu^*$. So $\text{UCB}_k(t) < \mu^* \leq \text{UCB}_{k^*}(t)$, meaning arm k is not chosen. $\square$

Bounding $\mathcal{E}_1$ and $\mathcal{E}_2$ via Hoeffding

Events $\mathcal{E}_1$ and $\mathcal{E}_2$ are confidence interval failure events. Hoeffding's inequality bounds each directly.

For any fixed n samples and any $s > 0$, Hoeffding gives (with $\delta = \sqrt{2s/n}$):

$$P\left(Q_k - \mu_k \geq \sqrt{\frac{2s}{n}}\right) \leq e^{-2n\delta^2} = e^{-2n \cdot \frac{2s}{n}} = e^{-4s}$$

Taking a union bound over *all* possible values $n = 1, 2, \dots, t$ and setting $s = \ln t$:

$$P(\mathcal{E}_2(t)) \leq \sum_{n=1}^t P\left(Q_k - \mu_k \geq \sqrt{\frac{2 \ln t}{n}} \mid N_k = n\right) \leq \sum_{n=1}^t e^{-4 \ln t} = \sum_{n=1}^t \frac{1}{t^4} = \frac{1}{t^3}$$

Why $\alpha = t^{-4}$?

Choosing the bonus $\sqrt{2 \ln t / N_k(t)}$ corresponds to $\alpha = t^{-4}$ in our Hoeffding setup. Each confidence interval failure event then has probability $\leq t^{-4}$ per arm per sample count. After union-bounding over all $n = 1, \dots, t$ this gives $\leq t^{-3}$ per timestep, which is summable over T .

Event $\mathcal{E}_3$: Complete Proof

We now bound the expected number of times arm k is pulled. Decompose:

$$\mathbb{E}[N_k(T)] = \underbrace{\ell}_{\text{pulls when } \mathcal{E}_3 \text{ holds}} + \sum_{t=K+1}^T P(A_t = k, N_k(t) > \ell)$$

By the Key Lemma, $\{A_t = k\} \cap \{\mathcal{E}_3 \text{ fails}\} \subseteq \mathcal{E}_1(t) \cup \mathcal{E}_2(t)$. Therefore:

$$P(A_t = k, N_k(t) > \ell) \leq P(\mathcal{E}_1(t)) + P(\mathcal{E}_2(t)) \leq \frac{1}{t^3} + \frac{1}{t^3} = \frac{2}{t^3}$$

Since $2/t^3 \leq 2/t^2$ for all $t \geq 1$, summing over all timesteps:

$$\sum_{t=K+1}^T P(A_t = k, N_k(t) > \ell) \leq \sum_{t=1}^T \frac{2}{t^2} \leq \sum_{t=1}^{\infty} \frac{2}{t^2} = \frac{\pi^2}{3}$$

Therefore:

$$\mathbb{E}[N_k(T)] \leq \ell + \frac{\pi^2}{3} = \left\lceil \frac{8 \ln T}{\Delta_k^2} \right\rceil + \frac{\pi^2}{3}$$

Assembling the UCB1 Regret Bound

Recall the regret decomposition:

$$\mathbb{E}[R_T] = \sum_{k: \Delta_k > 0} \Delta_k \cdot \mathbb{E}[N_k(T)]$$

Substituting our bound $\mathbb{E}[N_k(T)] \leq \frac{8 \ln T}{\Delta_k^2} + 1 + \frac{\pi^2}{3}$:

$$\begin{aligned} \mathbb{E}[R_T] &\leq \sum_{k: \Delta_k > 0} \Delta_k \left(\frac{8 \ln T}{\Delta_k^2} + 1 + \frac{\pi^2}{3} \right) \\ &= \sum_{k: \Delta_k > 0} \left(\frac{8 \ln T}{\Delta_k} + \left(1 + \frac{\pi^2}{3} \right) \Delta_k \right) \end{aligned}$$

UCB1 Regret (Final)

$$\mathbb{E}[R_T] \leq \sum_{k: \Delta_k > 0} \frac{8 \ln T}{\Delta_k} + \left(1 + \frac{\pi^2}{3} \right) \sum_{k: \Delta_k > 0} \Delta_k = O(\log T) \quad \square$$

UCB1 achieves $O(\log T)$ regret with instance-dependent constants that depend on the gaps Δ_k . This matches the Lai-Robbins lower bound up to constants, making UCB1 an asymptotically optimal algorithm.

Algorithm	Regret	No-Regret?	Key Idea
Pure Greedy	$O(T)$	×	Exploit only
ϵ -Greedy (fixed)	$O(T)$	×	Explore randomly
ϵ_t -Greedy ($1/t$)	$O(\log T)$	✓	Decay exploration
UCB1	$O(\log T)$	✓	Directed exploration
<i>Lower Bound</i>	$\Omega(\log T)$	—	Lai & Robbins (1985)

The key lesson: The exploration-exploitation trade-off cannot be resolved by ignoring uncertainty (Greedy) or randomising it away (ϵ -Greedy). It requires a principled accounting of uncertainty via confidence bounds.

Open Question for Students

UCB1 requires us to know the reward range $[0, 1]$. What if rewards are unbounded? What if the reward distributions are non-stationary (change over time)?